Pokročilé objektové technológie

7

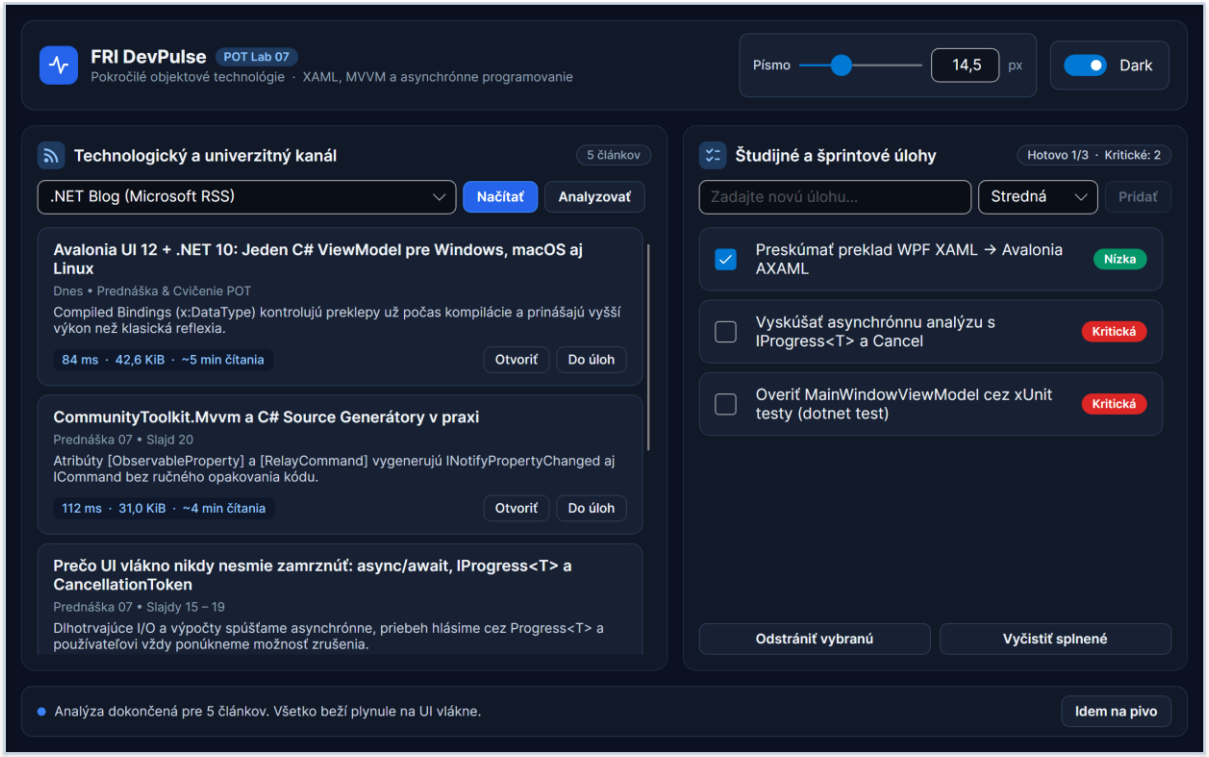
GUI – XAML, WPF & Avalonia UI

Architektúra MVVM a asynchrónne rozhranie v .NET 10
s využitím AI agentov

KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Cieľ cvičenia – aplikácia FRI DevPulse (Lab 07)

- **Dokončíte multiplatformovú desktopovú aplikáciu FRI DevPulse** (Windows, macOS, Linux):
  - **Úloha U1 (Ručne):** Kontajnery, CSS-like štýly, téma a obojsmerný **Element Binding**
  - **Úloha U2 (Ručne -> S AI):** Správca úloh cez **CommunityToolkit.Mvvm** a **IValueConverter**
  - **Úloha U3 (S AI agentom):** RSS/Atom čítačka + async analýza (**IProgress<T>**, **CancellationToken**)
  - **Úloha U4 (S AI agentom):** Automatizované **xUnit testy ViewModelu** bez okna + mutačná skúška
  - **Bonus (voliteľne):** Animované tlačidlo **„Idem na pivo“** prepojené so stavom MVVM
- **Podrobný sprievodca:** Celý postup nájdete aj v dokumente `POT_Lab07_Navod.docx`.

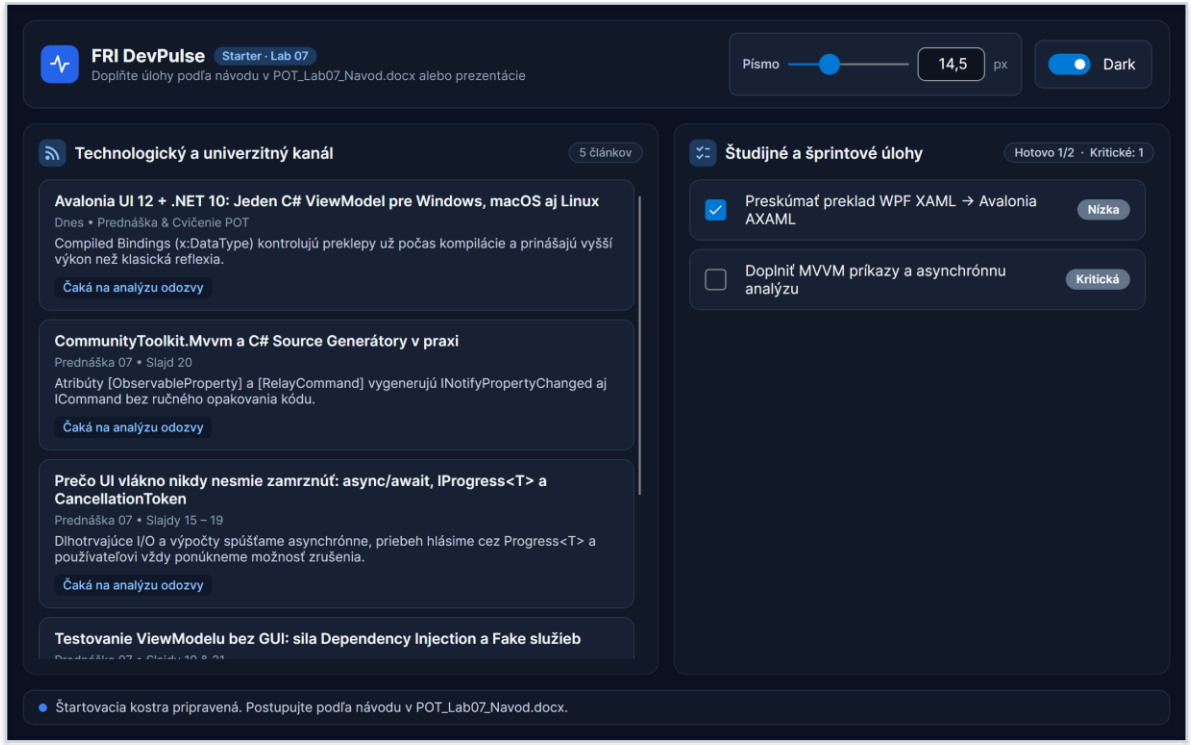


Výsledná podoba aplikácie FRI DevPulse (Avalonia UI 12 · Tmavá téma)

Príprava a spustenie (Windows / macOS / Linux)

- Rozbaľte **`POT_Lab07_Starter_NET10.zip`** a otvorte **`DevPulse.slnx`**:
  - **Visual Studio 2026 (Windows):** stlačte rovno **F5** – projekt **`DevPulse.App`** je prvý v riešení a spustí sa **aj bez nainštalovaného rozšírenia Avalonia**
  - **Rider / VS Code / Antigravity / Cursor:** natívne na Windows, macOS aj Linuxe
- Overenie zostavenia, úvodného testu a spustenie z terminálu:

```
# Overenie prostredia a spustenie štartovacej kostry:
dotnet restore DevPulse.slnx
dotnet build DevPulse.slnx
dotnet test DevPulse.slnx
dotnet run --project DevPulse.App
```



Východiskový stav po spustení štartovacej kostry DevPulse.App

Mapa projektu Starter – 5 upravovaných súborov

Súbor v štartovacom riešení	Vrstva	Čo v ňom dnes doplníte (hľadajte komentár TODO)
`DevPulse.App/Views/MainWindow.axaml`	View (UI)	U1 #ZoomSlider, U2 TODO formulár a $parent[Window], U3 panel a ProgressBar
`DevPulse.App/App.axaml`	View (Štýly)	U1 plynulý prechod Transitions a hover štýl Border.card:pointerover
`DevPulse.Core/ViewModels/MainWindowViewModel.cs`	ViewModel	U2 príkazy úloh a IDialogService, U3 LoadFeedAsync a AnalyzeFeedAsync
`DevPulse.App/Converters/PriorityToBrushConverter.cs`	View (Konvertor)	U2 prevod enumu TaskPriority na farbu odznaku (IValueConverter)
`DevPulse.Tests/MainWindowViewModelTests.cs`	xUnit Testy	U4 automatizované testy ViewModelu s Fake službami bez spúšťania okna

TIP: Hotový Composition Root a užitočné klávesové skratky

Dependency Injection v `DevPulse.App/App.axaml.cs` je už zapojené (`Host.CreateApplicationBuilder()`). · **Prechod do vygenerovaného *.g.cs kódu:** `F12` (macOS: `Cmd+Click` / `Cmd+B`) · **Avalonia DevTools v bežiacom okne:** `F12` (macOS: `Fn+F12`).

Prekladový prehľad: WPF -> Avalonia UI 12

Koncept	WPF (Prednáška 07)	Avalonia UI 12 (Lab 07)	Výhoda / Na čo si dať pozor
Súbor pohľadu	`MainWindow.xaml`	`MainWindow.axaml`	C# ViewModel v DevPulse.Core je 100 % spoločný!
Kontrola bindingu	Reflexia za behu	`x:DataType="vm:..."`	Compiled Bindings odhalia preklep pri builde (AVLN2000)
Binding na prvok	`{Binding Value, ElementName=s}`	`{Binding #s.Value}`	Kratší a čitateľnejší zápis priamo v XAML
Skrytie prvku	Visibility="Collapsed"	`IsVisible="{Binding IsBusy}"`	Priamo bool bez BooleanToVisibilityConverter
TextBox vstup	Predvolene LostFocus	Predvolene PropertyChanged	Pri rozsahu Slidera dajte LostFocus; zástupný text: PlaceholderText
Štýly a prechody	Style TargetType + Storyboard	Selector="Border.card:pointerover"	CSS selektory + deklaratívne <Transitions>

Práca s AI asistentom – kontext AGENTS.md

- **Prečo AI pri Avalonii bez architektonického kontextu často chybuje?**
  - Jazykové modely majú v tréningovom korpuse násobne viac starého WPF kódu než Avalonie 12
  - Bez pravidiel AI generuje `Visibility="Collapsed"`, zastaraný `Watermark` namiesto `PlaceholderText`, zabúda na `x:DataType` a volá `MessageBox.Show` vo ViewModeli
- **Ako nastaviť AI na cvičení:**
  - Prečítajte si súbory `AGENTS.md` a `.github/copilot-instructions.md` v koreni projektu
  - Copilot, Antigravity a Cursor ich načítajú automaticky; do webového chatu pripojte obsah `AGENTS.md` na začiatku

ODPORÚČANÝ POSTUP: Kedy písať ručne a kedy zapojiť AI

Úlohu U1 a prvý príkaz AddTask v U2 napíšte ručne bez AI – vyskúšate si kontrolu kompilátora `AVLN2000` aj prechod cez `F12` do vygenerovaného kódu. **Od druhej časti U2 zapojte AI asistenta** a kontrolujte čistotu hranice MVVM (`DevPulse.Core` nesmie poznať `Avalonia.*`).

U1 (časť A) – Obojsmerný Element Binding

RUČNE BEZ AI · 10 MIN

- V `DevPulse.App/Views/MainWindow.axaml` previažte `ZoomSlider` (`Minimum="12"`) a vedľajší **TextBox** **obojsmerne** s `UpdateSourceTrigger=LostFocus`:
  - Po napísaní čísla `18` stlačte **Tab** (bez `LostFocus` by prvá číslica `1` hneď skočila na `12,0`!)
  - K obom existujúcim `<TextBlock ... Text="{Binding Title}">` pridajte atribút `FontSize="{Binding #ZoomSlider.Value}"` (pri úlohe vpravo **zachovajte** `Grid.Column="1"`!)
  - Vyskúšajte zámerný preklep `{Binding Titulok}` -> overte chybu prekladu `AVLN2000`

```
<!-- MainWindow.axaml – obojsmerný Element Binding v Avalonia UI 12 -->
<Slider x:Name="ZoomSlider" Width="115" Minimum="12" Maximum="20" Value="14.5"
        TickFrequency="0.5" IsSnapToTickEnabled="True" />
<TextBox Width="56" HorizontalContentAlignment="Center"
         Text="{Binding #ZoomSlider.Value, Mode=TwoWay, UpdateSourceTrigger=LostFocus, StringFormat={}{0:F1}}" />

<!-- Doplňte atribút FontSize k obom nadpisom kartičiek (pri úlohe nechajte Grid.Column="1"): -->
<TextBlock Grid.Column="1" Text="{Binding Title}" FontSize="{Binding #ZoomSlider.Value}" ... />
```

U1 (časť B) – CSS-like štýly, Transitions a téma

RUČNE BEZ AI · 5 MIN

- Otvorte súbor **`DevPulse.App/App.axaml`**
- V existujúcom štýle **`Border.card`** **ponechajte všetkých 6 pôvodných Setterov** (zaoblenie, okraje, pozadie) a doplňte **Transitions**:
  - Pod neho pridajte hover selektor **Border.card:pointerover** – karta sa jemne nadvihne o **`-2px`** a zvýrazní okraj cez **`BrandAccentBrush`**
- Spustite aplikáciu, prepnite tému **Dark / Light** a preskúmajte strom prvkov cez **F12** (macOS: **Fn+F12**)

```
<!-- App.axaml -->
<Style Selector="Border.card">
    <!-- Ponechajte pôvodných 6 Setterov! -->
    <Setter Property="RenderTransform"
            Value="translateY(0px)" />
    <Setter Property="Transitions">
        <Transitions>
            <TransformOperationsTransition
                Property="RenderTransform"
                Duration="0:0:0.16" />
        </Transitions>
    </Setter>
</Style>

<Style Selector="Border.card:pointerover">
    <Setter Property="BorderBrush"
            Value="{DynamicResource BrandAccentBrush}" />
    <Setter Property="Background"
            Value="{DynamicResource CardHoverBgBrush}" />
    <Setter Property="RenderTransform"
            Value="translateY(-2px)" />
</Style>
```

U2 (časť A) – Príkazy v CommunityToolkit.Mvvm

ADDTASK RUČNE -> ZVYŠOK S AI

- Otvorte **MainWindowViewModel.cs** (metóda **CreateTask** je pripravená):
  - **Ručne:** Nad **newTaskTitle** pridajte **[NotifyCanExecuteChangedFor(nameof(AddTaskCommand))]** a dopíšte **CanAddTask()** + **AddTask()**
  - **Inšpekcia generátora:** Zostavte riešenie a cez **F12** (macOS: **Cmd+Click**) si prezrite vygenerované ***.g.cs** súbory
  - **S AI asistentom:** Doplňte **AddArticleToTasks(FeedArticle?)**, **RemoveSelectedTask()** a **ClearCompletedTasksAsync()** cez **await _dialogService.ConfirmAsync(...)**

```
// MainWindowViewModel.cs (partial trieda!)
[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof(AddTaskCommand))]
private string newTaskTitle = "";

private bool CanAddTask()
    => !string.IsNullOrWhiteSpace(NewTaskTitle);

[RelayCommand(CanExecute = nameof(CanAddTask))]
private void AddTask()
{
    var task = CreateTask(
        NewTaskTitle.Trim(), SelectedPriority);
    Tasks.Insert(0, task);
    NewTaskTitle = "";
}

[RelayCommand]
private async Task ClearCompletedTasksAsync()
{
    var confirmed = await _dialogService.ConfirmAsync(
        "Vyčistiť splnené úlohy", "Odstrániť splnené?");
    if (!confirmed) return;
    foreach (var t in Tasks.Where(x => x.IsCompleted).ToList())
        Tasks.Remove(t);
}
```

U2 (časť B) – Konvertor farieb a $parent[Window]

S AI AGENTOM + CODE REVIEW

- **1. PriorityToBrushConverter.cs:** `Nizka` -> `#059669`, `Stredna` -> `#D97706`, `Kriticka` -> `#DC2626`
- **2. Pravý panel v MainWindow.axaml:** Vložte vstupný riadok (`NewTaskTitle` s `PlaceholderText`, `ComboBox` priorít, tlačidlo `Pridať` s `Classes="accent"` a `IsDefault="True"`). Do `<UniformGrid DockPanel.Dock="Bottom">` (**musí ostať pred <ListBox>!**) vložte `Odstrániť vybranú` a `Vyčistiť splnené`
- **3. Šablóna článku:** Tlačidlá v `DataTemplate` článku volajú ViewModel okna cez **$parent[Window]**:

```
<!-- V DataTemplate článku (models:FeedArticle) vnútri Grid ColumnDefinitions="*,Auto,Auto": -->
<Button Grid.Column="1" Classes="subtle" Content="Otvoriť"
        Command="{Binding $parent[Window].((vm:MainWindowViewModel)DataContext).OpenArticleCommand}"
        CommandParameter="{Binding Link}" />

<Button Grid.Column="2" Classes="subtle" Content="Do úloh"
        Command="{Binding $parent[Window].((vm:MainWindowViewModel)DataContext).AddArticleToTasksCommand}"
        CommandParameter="{Binding}" />
```

U3 (časť A) – Asynchrónny kanál a IProgress<T>

S AI AGENTOM + CODE REVIEW

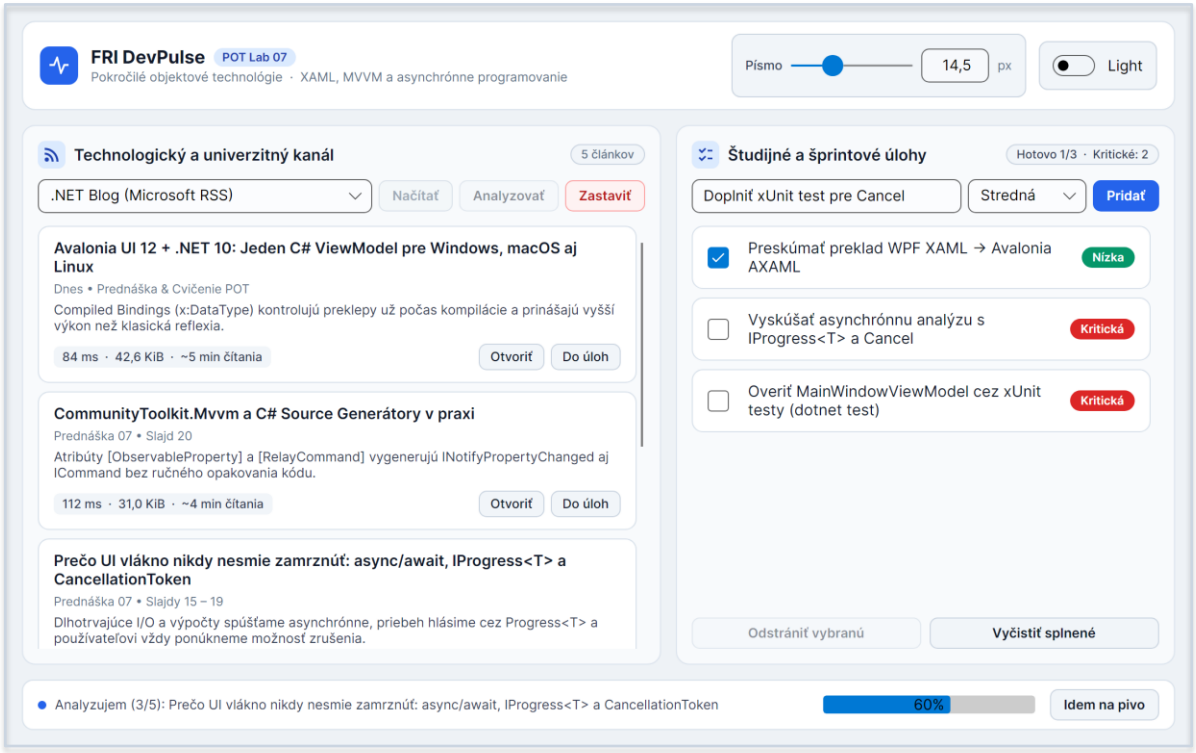
- Počas sťahovania **nesmie okno zamrznúť**! Nad `isBusy` pridajte `[NotifyCanExecuteChangedFor]` pre `LoadFeedCommand` aj `AnalyzeFeedCommand`
- Implementujte `LoadFeedAsync(CancellationToken)` a `AnalyzeFeedAsync(CancellationToken)` s **IncludeCancelCommand = true**:
  - Služba `_feedService.AnalyzeArticlesAsync(snapshot, progress, token)` hlási postup z pracovného vlákna cez **IProgress<AnalysisProgress>** (lokálna stráž `lock (gate)` + `isActive` chráni záverečný status aj v `xUnit` testoch)

```
private bool CanAnalyzeFeed() => Articles.Count > 0 && !IsBusy;
[RelayCommand(CanExecute = nameof(CanAnalyzeFeed), IncludeCancelCommand = true)]
private async Task AnalyzeFeedAsync(CancellationToken token) {
    if (Articles.Count == 0) return;
    var snapshot = Articles.ToList(); var gate = new object(); var isActive = true;
    IsBusy = true; ProgressPercent = 0;
    IProgress<AnalysisProgress> progress = new Progress<AnalysisProgress>(p => {
        lock (gate) {
            if (!isActive || token.IsCancellationRequested) return;
            ProgressPercent = p.Percent;
            StatusMessage = $"Analyzujem ({p.Completed}/{p.Total}): {p.CurrentTitle}";
        }
    });
    try {
        await _feedService.AnalyzeArticlesAsync(snapshot, progress, token);
        lock (gate) { isActive = false; ProgressPercent = 100; StatusMessage = "Analýza dokončená."; }
    } catch (OperationCanceledException) {
        lock (gate) { isActive = false; StatusMessage = "Analýza bola zastavená."; }
    } finally { lock (gate) isActive = false; IsBusy = false; }
}
```

U3 (časť B) – ProgressBar a zrušenie operácie

S AI AGENTOM + ŽIVÝ TEST

- Do ľavého panela v `MainWindow.axaml` doplňte:
  - `ComboBox` (`Sources` / `SelectedSource`) + tlačidlá **Načítať** (`Classes="accent"`, `LoadFeedCommand`) a **Analyzovať** (`AnalyzeFeedCommand`)
  - Tlačidlo **Zastaviť** (`Classes="danger"`) s `AnalyzeFeedCancelCommand` a `IsVisible="{Binding AnalyzeFeedCommand.IsRunning}"`
- Do spodného riadku vložte `ProgressBar` s `Grid.Column="1"` `Width="200"` (`Value="{Binding ProgressPercent}"`, `IsVisible="{Binding IsBusy}"`)
- **Vyskúšajte:** Počas analýzy hýbte oknom, prepnite tému a otestujte okamžité prerušenie tlačidlom **Zastaviť**



Bežiaca asynchrónna analýza (60 %) v svetlej téme – tlačidlo Zastaviť a živý ProgressBar

U4 – Automatické testy ViewModelu bez GUI (xUnit)

S AI AGENTOM + MUTAČNÁ SKÚŠKA

- Otvorte projekt **DevPulse.Tests** (**Fakes.cs** a **MainWindowViewModelTests.cs**)
- Doplňte **xUnit testy** s **FakeFeedService** a **FakeDialogService**:
  - Zakázané / povolené **AddTaskCommand.CanExecute** a udalosť **CanExecuteChanged**
  - Potvrdenie verzus zamietnutie mazania v **FakeDialogService**
  - Dokončenie analýzy (**IsAnalyzed**) a zrušenie cez **AnalyzeFeedCancelCommand**
- **Mutačná skúška:** Zakomentujte celú vetvu **if (!confirmed)** (aj s blokom **{ ... }**) a overte, že **dotnet test** okamžite zlyhá!

```
[Fact]
public async Task ClearCompleted_AkZrusene_NezmazeNic()
{
    var dialogs = new FakeDialogService
    {
        NextConfirmationResult = false
    };
    var vm = new MainWindowViewModel(
        new FakeFeedService(),
        dialogs,
        new FakeBrowserLauncher());
    var pred = vm.Tasks.Count;

    await vm.ClearCompletedTasksCommand.ExecuteAsync(null);

    Assert.Equal(1, dialogs.ConfirmCallsCount);
    Assert.Equal(pred, vm.Tasks.Count);
}

// Spustenie v termináli (zbehne < 100 ms bez okna!):
// > dotnet build DevPulse.slnx
// > dotnet test DevPulse.slnx
```

Voliteľný Bonus – tlačidlo „Idem na pivo“ v MVVM

- Pre rýchlejších: preklopte ukážku zo slajdu 14 prednášky WPF do **MVVM stavu + plynulej animácie**:
  - Vo ViewModeli pridajte vlastnosť `HasPendingCriticalTasks` (`PendingCriticalCount > 0`) a príkaz `GoForBeerCommand`
  - Do stavového riadka (`Grid.Column="2"`) pridajte tlačidlo **Idem na pivo** s `TranslateTransform` a `DoubleTransition`
  - Pri udalosti `PointerEntered` skontrolujte `vm.HasPendingCriticalTasks` – kým ostávajú kritické úlohy, tlačidlo uskočí o `-130 px`
  - Po odškrtnutí všetkých kritických úloh tlačidlo zostane stáť a cez `IDialogService` zobrazí pochvalu

```xaml
<!-- MainWindow.axaml (Grid.Column="2") -->
<Button Grid.Column="2" Content="Idem na pivo"
        Command="{Binding GoForBeerCommand}"
        PointerEntered="BeerButton_OnPointerEntered">
  <Button.RenderTransform>
    <TranslateTransform X="0">
      <TranslateTransform.Transitions>
        <Transitions>
          <DoubleTransition Property="X" Duration="0:0:0.18" />
        </Transitions>
      </TranslateTransform.Transitions>
    </TranslateTransform>
  </Button.RenderTransform>
</Button>
```

```csharp
// MainWindow.axaml.cs – čisto vizuálna reakcia View
private void BeerButton_OnPointerEntered(
    object? sender, PointerEventArgs e)
{
    if (DataContext is MainWindowViewModel vm &&
        sender is Button {
            RenderTransform: TranslateTransform t
        })
    {
        t.X = vm.HasPendingCriticalTasks
            ? (Math.Abs(t.X) < 10 ? -130 : 0)
            : 0;
    }
}
```

Riešenie najčastejších chýb na cvičení

Príznak / Kód chyby	Prečo vznikla	Ako ju rýchlo opraviť
AVLN2000: Unable to resolve property...	Preklep v {Binding}, chýba x:DataType alebo voláte príkaz okna z DataTemplate priamo.	Skontrolujte názov vlastnosti, pridajte x:DataType a v šablóne článku použite $parent[Window].((vm:MainWindowViewModel)DataContext).
CS0103: AddTaskCommand does not exist	Source Generator nevygeneroval druhú časť triedy alebo sa nezhoduje názov poľa.	Overte public partial class MainWindowViewModel, atribút [RelayCommand] nad metódou AddTask() a malé písmeno v private string newTaskTitle.
Zoznam nevypĺňa okno / prekrytý text	Nesprávne poradie v DockPaneli (LastChildFill) alebo chýba Grid.Column.	V DockPaneli musí byť <ListBox> zapísaný ÚPLNE POSLEDNÝ (až pod DockPanel.Dock="Bottom")! Na ProgressBar pridajte Grid.Column="1".

TIP: Do poľa veľkosti písma nejde napísať 18 (skočí na 12,0)?

Do väzby pridajte `UpdateSourceTrigger=LostFocus` a po napísaní čísla stlačte `Tab` (alebo kliknite mimo poľa), aby `ZoomSlider` (`Minimum="12"`) neorezal hodnotu už po stlačení prvej číslice `1`.

Vlastná kontrola a založenie vlastného projektu

- **1. Obojsmerné previazanie:** Zmena čísla v `TextBox`-e (+ `Tab`) hýbe posuvníkom a mení veľkosť písma kartičiek
- **2. Čistá MVVM architektúra:** Projekt `DevPulse.Core` nemá žiadnu referenciu na `Avalonia.*` ani `System.Windows.*`
- **3. Neblokované UI vlákno:** Počas analýzy beží `ProgressBar`, okno reaguje a tlačidlo **Zastaviť** okamžite preruší beh
- **4. Zelené unit testy + porozumenie:** `dotnet test` prechádza a viete vysvetliť `x:DataType`, `partial`, `IProgress<T>` a `IDialogService`

```
# Založenie novej Avalonia MVVM aplikácie od nuly (napr. na semestrálnu prácu):
dotnet new install Avalonia.Templates
dotnet new avalonia.mvvm -n MojaSemestralka -f net10.0 -m CommunityToolkit
cd MojaSemestralka && dotnet run
```

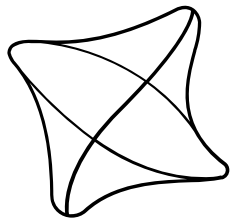
Zdroje a dokumentácia

- **Oficiálna dokumentácia Avalonia UI 12:** https://docs.avaloniaui.net/
- **CommunityToolkit.Mvvm (Source Generátory):** https://learn.microsoft.com/dotnet/communitytoolkit/mvvm/
- **Oficiálna dokumentácia Microsoft WPF (.NET 10):** https://learn.microsoft.com/dotnet/desktop/wpf/
- **Galéria ukážok Avalonia UI:** https://github.com/AvaloniaUI/Avalonia.Samples
- **Prednáška 07 (prof. Ing. Emil Kršák, PhD.):** projekty `GUI-WPF-ukazky-v2` a `GUI-MVVM-ukazka-v2`

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu **Pokročilé objektové technológie** na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline

- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Ing. **Štefan Toth**, PhD.
Katedra softvérových technológií
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
stefan.toth@fri.uniza.sk